

# Catching Campaigns, Not Connections: Coordination-Aware Hypergraph Networks for Multi-Stage Intrusion Detection

Anonymous Authors  
*Institution*

## Abstract

Provenance-based intrusion detection systems analyze kernel-level audit logs to identify Advanced Persistent Threats, but existing approaches operate at coarse granularities — classifying entire graphs (KAIROS, MAGIC) or individual nodes (FLASH, ThreaTrace) — without attributing detection to specific attack events. We present HyperMamba, a system that detects individual crossprocess attack events in real-time audit streams by maintaining persistent entity state via Selective State Space Models. HyperMamba models each audit event as a temporal hyperedge connecting two or three system entities, preserving the native variable-arity structure of the Common Data Model. A Selective SSM maintains a state vector for every entity, accumulating behavioral context across the entire stream; at each event, attention-based hyperedge aggregation combines participating entities’ states, the SSM updates each entity, and a classifier produces an event-level detection decision. We prove empirically that this accumulated state is the primary detection mechanism: an ablation that removes persistent state while retaining the complete event encoder and classifier collapses test AUPRC from 0.73 to 0.33. A gated process identity modulation mechanism provides campaign-invariant semantic anchors, enabling generalization to entirely unseen attack campaigns. On the DARPA TC Engagement 3 Theia dataset, HyperMamba achieves event-level AUPRC of 0.86 under same-campaign continuation and 0.72 under cross-campaign generalization, with false positive rates of 0.000010 and 0.000195 respectively, while processing events at  $194\times$  the peak audit ingestion rate on a single GPU.

## 1 Introduction

Provenance-based Network Intrusion Detection Systems (NIDS) analyze kernel-level audit logs to detect Advanced Persistent Threats (APTs) on monitored hosts. Recent state-of-the-art systems — KAIROS [?], MAGIC [?], FLASH [?], and ThreaTrace [?] — have demonstrated strong detection capabilities. However, these systems operate at fundamentally coarse

granularities: KAIROS and MAGIC assign anomaly scores to entire provenance graphs, while FLASH and ThreaTrace classify individual nodes (processes or files). None produce *event-level* classifications. When a security analyst receives a node-level alert flagging a process as malicious, the analyst must still manually inspect thousands of that process’s events to identify which specific operations constituted the attack. This forensic gap between detection and attribution limits the operational utility of existing systems.

The granularity gap is not merely inconvenient — it reflects a structural limitation. Crossprocess attack events, executed by child processes spawned during multi-stage APT campaigns, use the same system calls as benign background activity. Their per-event feature distributions satisfy  $P(\mathbf{x}_i \mid \text{attack}) \approx P(\mathbf{x}_i \mid \text{benign})$ , making them statistically indistinguishable in isolation. A system that evaluates events independently cannot reliably separate malicious reads and writes from benign ones. The discriminative signal exists only in the *accumulated behavioral history* of the acting entity: the sequence of prior operations that reveals the entity’s role in an ongoing campaign. Existing systems discard this signal by operating on static graph snapshots without persistent entity state.

We present **HyperMamba**, a provenance-based NIDS that detects individual crossprocess attack events in real-time audit streams by maintaining persistent entity state via Selective State Space Models. HyperMamba models each audit event as a temporal hyperedge connecting two or three system entities, preserving the native variable-arity structure of the Common Data Model (CDM). A Selective SSM maintains and updates a state vector for every entity in the system, accumulating behavioral context across the entire event stream. At each event, the model aggregates the participating entities’ states via attention-based hyperedge aggregation, updates each entity’s state through the SSM recurrence, and classifies the event using both the updated states and the event’s own features. The result is event-level, real-time detection with per-event forensic attribution.

Our contributions are:

1. **Provenance hypergraph formalization (C1).** We formalize audit streams as temporal hypergraphs with variable-arity hyperedges and demonstrate via controlled ablation that cross-entity state aggregation within hyperedges provides - AUPRC improvement over isolated per-entity tracking.
2. **Persistent entity state as the detection mechanism (C2).** We prove empirically that accumulated entity state — not isolated event features — is the primary mechanism enabling crossprocess attack detection. Removing persistent state collapses test AUPRC from 0.73 to 0.33 (Section 5.3).
3. **Gated process identity for cross-campaign generalization (C3).** We introduce a gated process-path modulation mechanism that provides campaign-invariant semantic anchors, enabling detection to generalize to entirely unseen attack campaigns (Section 5.5).
4. **Event-level detection at operational scale (C4).** HyperMamba achieves event-level AUPRC of 0.86 (same-campaign) and 0.72 (cross-campaign) with FPR of 0.000010 and 0.000195 respectively, while processing 194× the peak audit ingestion rate on a single GPU (Section 5.2).

Section 2 introduces the provenance data model and SSM background. Section 3 defines the threat model and detection task. Section 4 details the HyperMamba architecture. Section 5 presents the evaluation. Sections 7 and 6 discuss related work and limitations.

## 2 Background

This section defines the three concepts required to follow the architecture (Section 4): the provenance data model, its formalization as a temporal hypergraph, and the Selective State Space Model recurrence.

### 2.1 Provenance Data Model

Modern host-based audit systems based on the DARPA Common Data Model (CDM) record every kernel-mediated operation as a typed event linking a *subject* (the acting process) to one or two *objects* (the targets of the operation). A file read generates an event with one subject and one object (process → file); a memory mapping of a shared library generates an event with one subject and two objects (process → file → memory\_object). Each event carries metadata: a discrete event type (one of ~40 CDM types such as EVENT\_READ, EVENT\_MMAP), continuous attributes (byte count, flags), and a nanosecond-resolution timestamp. Over a multi-day engagement, a single host generates tens of millions of such events.

### 2.2 Provenance Hypergraph

We formalize the audit stream as a temporal hypergraph.

**Definition 1** (Provenance Hypergraph). A provenance hypergraph is a tuple  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , where  $\mathcal{V}$  is a finite set of system entities (processes, files, sockets, memory objects) and  $\mathcal{E} = \{e_1, e_2, \dots\}$  is a chronologically ordered sequence of hyperedges. Each hyperedge  $e_t = (v_{subj}, v_{obj}, v_{obj2}, \tau_t, \mathbf{a}_t)$  connects  $|V_{e_t}| \in \{2, 3\}$  entities, where  $v_{obj2}$  is absent for size-2 events,  $\tau_t$  is the event timestamp, and  $\mathbf{a}_t$  is the attribute vector.

For example, when the compromised Firefox process maps the dropped binary `malicious.so` into executable memory, the resulting EVENT\_MMAP hyperedge connects three entities: Firefox ( $v_{subj}$ ), the file `malicious.so` ( $v_{obj}$ ), and the allocated memory region ( $v_{obj2}$ ).

**Why hyperedges, not pairwise edges.** Prior provenance-based NIDS [?, ?, ?, ?] decompose each multi-object event into independent pairwise edges, discarding the co-occurrence structure. In the DARPA TC E3 Theia dataset, 2.8% of events are size-3 hyperedges. While this fraction appears small, size-3 events are 3.0× enriched among crossprocess attack events relative to benign events, driven primarily by EVENT\_MMAP operations (e.g., a compromised process mapping a malicious shared library into executable memory). This enrichment motivates retaining the native hyperedge arity rather than decomposing into pairwise interactions.

### 2.3 Selective State Space Models

State Space Models (SSMs) define a linear recurrence over a latent state  $\mathbf{h} \in \mathbb{R}^n$ , mapping an input sequence  $x(t)$  to an output  $y(t)$  through continuous-time dynamics:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}\mathbf{h}(t) \quad (1)$$

where  $\mathbf{A} \in \mathbb{R}^{n \times n}$  governs state decay and  $\mathbf{B} \in \mathbb{R}^{n \times 1}$  controls input injection. For discrete sequences, the continuous dynamics are discretized with a step size  $\Delta$  to yield:

$$\bar{\mathbf{A}} = \exp(\Delta\mathbf{A}), \quad \bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\bar{\mathbf{A}} - \mathbf{I}) \cdot \Delta\mathbf{B} \quad (2)$$

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}x_t, \quad y_t = \mathbf{C}\mathbf{h}_t \quad (3)$$

The *Selective* SSM [?] makes  $\mathbf{B}$ ,  $\mathbf{C}$ , and  $\Delta$  input-dependent, allowing the model to selectively retain or discard information at each step based on the current input. In Section 4.5, we adapt this mechanism to operate on entity states within a hypergraph, where  $\Delta$  is further conditioned on the elapsed time since the entity was last observed.

## 3 Threat Model

### 3.1 Attacker Model

We consider an Advanced Persistent Threat (APT) adversary executing multi-stage crossprocess attack campaigns on a monitored host.

**Knowledge.** The attacker is aware that kernel-level audit logging is active on the target host. In the primary evaluation, the attacker does not have knowledge of the detection model’s architecture or parameters (black-box setting). We discuss the implications of a white-box attacker who targets the SSM state propagation mechanism in Section 6.

**Capabilities.** The attacker can execute arbitrary operations through standard system APIs: spawning processes, reading and writing files, mapping memory regions, and establishing network connections. All such operations are recorded as audit events by the kernel’s logging infrastructure. The attacker *cannot* tamper with or suppress kernel-level audit logs. This assumption is standard in the provenance-based NIDS literature and reflects deployments where audit collection runs at a higher privilege level than user-space processes.

**Goal.** The attacker’s objective is to execute a complete multi-stage crossprocess campaign—initial exploitation of an entry-point application, deployment of secondary tooling via child processes, establishment of command-and-control channels, and data exfiltration—while evading event-level detection. The attacker achieves evasion if the individual system events generated by the campaign’s child processes are not flagged as malicious by the detection system.

## 3.2 DARPA TC E3 Attack Instantiation

We ground the threat model in a specific multi-stage campaign from the DARPA TC Engagement 3 Theia dataset. The attack proceeds through four distinct phases, each producing audit events in the host’s kernel-level log:

1. **Initial compromise.** The attacker exploits a vulnerability in the Firefox browser process. Firefox executes an `EVENT_WRITE` to drop a malicious binary at `/home/admin/clean`. This single event is the *entry point*—Firefox is the compromised process.
2. **Payload activation.** Firefox spawns the `clean` process via `EVENT_EXECUTE`. The `clean` binary loads shared libraries (`EVENT_MMAP`) and reads system configuration files (`EVENT_READ` on `/etc/resolv.conf`, `/etc/hosts`).
3. **Command and control.** `clean` establishes a reverse shell connection to the attacker’s infrastructure via `EVENT_CONNECT` to an external IP address. Subsequent `EVENT_SENDTO` and `EVENT_RECVFROM` events carry C2 commands and responses.
4. **Exfiltration.** Under C2 direction, `clean` reads sensitive files (`EVENT_READ`) and transmits their contents to the attacker’s server (`EVENT_SENDTO`). Additional child processes may be spawned by `clean` to perform lateral reconnaissance.

**Crossprocess label definition.** In this paper, the *crossprocess* label refers to the events executed by the child processes spawned by the entry-point process—in this case, `clean` and any processes it subsequently spawns. The entry-point process itself (Firefox) is *excluded* from this label. This exclusion is deliberate: the initial exploit of Firefox may produce anomalous system calls (e.g., a browser writing an executable binary), which are detectable by simpler, stateless methods. The harder detection problem—and the one HyperMamba targets—is identifying the later-stage child processes whose individual events are operationally indistinguishable from benign system activity.

## 3.3 Why Crossprocess Attacks Are Hard for Stateless Detection

Crossprocess attacks are executed by *child processes* that the compromised entry-point process spawns to carry out the operational stages of the campaign—reconnaissance, lateral movement, data staging, and exfiltration. These child processes (e.g., a dropped binary `clean` or an injected shell script) use precisely the same system calls as benign background processes: `EVENT_READ` on configuration files, `EVENT_WRITE` to temporary directories, `EVENT_MMAP` for shared library loading, and `EVENT_CONNECT` to external hosts. A stateless classifier, which evaluates each event in isolation, observes these calls without historical context and faces a fundamental ambiguity.

**Concrete example.** Consider two events drawn from the DARPA TC E3 Theia dataset:

1. `bash` (benign) executes `EVENT_READ` on `/etc/resolv.conf`, transferring 242 bytes,  $\Delta t = 0.4s$  since its last event.
2. `clean` (crossprocess attack) executes `EVENT_READ` on `/etc/resolv.conf`, transferring 242 bytes,  $\Delta t = 0.3s$  since its last event.

Both events share the same event type, the same target file, and effectively identical continuous features (byte count, flags, inter-arrival time). A feature vector extracted from either event in isolation is statistically indistinguishable. More formally, the per-event feature distributions satisfy  $P(\mathbf{x}_i \mid \text{attack}) \approx P(\mathbf{x}_i \mid \text{benign})$  for the vast majority of crossprocess attack events. The discriminative signal is not in the event itself, but in the *accumulated behavioral history* of the subject entity: `clean` was spawned by a compromised Firefox process, previously established a C2 connection, and has been writing to staging directories—a pattern visible only through persistent state.

**Empirical validation.** The ablation study in Section 5.3 provides direct empirical evidence for this stealthiness condition. Removing accumulated entity state from HyperMamba (while retaining the complete event encoder, hyperedge aggregator, and classifier) causes the test AUPRC to collapse from

0.73 to 0.33. The training AUPRC also collapses from 0.86 to 0.51, confirming that the model cannot even fit the training distribution using local event features alone. Crossprocess attack events are, by construction, not identifiable without accumulated context.

### 3.4 Detection Task

Given a continuous chronological stream of audit events  $e_1, e_2, \dots$  generated by a single monitored host, the detection task is to classify each event  $e_t$  as either benign ( $y_t = 0$ ) or crossprocess-attack ( $y_t = 1$ ) at event arrival time. Formally, the detector is a function:

$$f_\theta(e_t | e_1, \dots, e_{t-1}) \rightarrow [0, 1] \quad (4)$$

that produces a probability  $\hat{y}_t$  using the current event  $e_t$  as input and all previously observed events  $e_1, \dots, e_{t-1}$  as historical context. The detector *cannot* access future events  $e_{t+1}, e_{t+2}, \dots$  — the classification must be strictly causal.

**Scope.** This work addresses host-based detection from kernel-level audit logs. Network-flow-based intrusion detection, payload inspection, and signature-based matching are outside the scope of this paper.

## 4 The HyperMamba Architecture

HyperMamba processes audit events in strict chronological order as a continuous stream. Each event is a temporal hyper-edge connecting two or three system entities (Section 2.2). For each incoming event, the model executes a five-stage forward pass that reads and updates persistent entity states. Figure ?? summarizes the pipeline; Sections 4.2–4.7 detail each stage.

### 4.1 Overview

Let  $e_t$  denote the audit event arriving at discrete step  $t$ , with participating entities  $V_{e_t} \subseteq \mathcal{V}$ , where  $|V_{e_t}| \in \{2, 3\}$ . We denote the three entity slots as  $v_{\text{subj}}$  (subject process),  $v_{\text{obj}}$  (primary object), and  $v_{\text{obj2}}$  (secondary object, absent for size-2 events). Each entity  $v \in \mathcal{V}$  maintains a persistent state vector  $\mathbf{s}_v \in \mathbb{R}^d$  in the EntityStateBank, where  $d = 128$  throughout this paper.

HyperMamba processes  $e_t$  in five stages:

1. **Event Encoding** (Section 4.2). The event type, continuous features (byte count, flags, temporal inter-arrival statistics), and the subject’s process path are projected and combined into a dense event representation  $\mathbf{f}_e \in \mathbb{R}^{2d}$ . Process path identity is injected via a gated additive residual with training-time identity dropout to prevent per-process memorization (Section 4.2).
2. **Entity State Retrieval** (Section 4.3). For each entity  $v \in V_{e_t}$ , the current state  $\mathbf{s}_v$  and elapsed time  $\Delta t_v$  since  $v$

was last observed are read from the EntityStateBank. A validity mask  $m_v$  marks absent secondary objects.

3. **Hyperedge Aggregation**,  $\mathcal{V} \rightarrow \mathcal{E}$  (Section 4.4). The retrieved entity states are aggregated into a single hyperedge representation  $\mathbf{x}_e \in \mathbb{R}^d$  via scaled dot-product attention, where event features  $\mathbf{f}_e$  serve as the query and the entity states — concatenated with learned role embeddings  $\mathbf{r}_v$  and log-scaled elapsed time  $\log(1 + \Delta t_v)$  — serve as keys and values.
4. **SSM State Update**,  $\mathcal{E} \rightarrow \mathcal{V}$  (Section 4.5). Each participating entity’s state  $\mathbf{s}_v$  is updated via a Selective State Space Model. The update is role-specific (subjects and objects receive asymmetric updates), time-aware (the discretization step  $\Delta_i$  incorporates  $\Delta t_v$ ), and gated (a learned residual gate controls how much of the SSM output replaces the prior state).
5. **Classification** (Section 4.6). The post-update entity states are mean-pooled (masked by validity) and concatenated with  $\mathbf{f}_e$  to produce a scalar logit  $\hat{y}_t = \sigma(\text{MLP}([\bar{\mathbf{s}} \parallel \mathbf{W}_p \mathbf{f}_e]))$ , where  $\hat{y}_t \in [0, 1]$  is the probability that  $e_t$  is a crossprocess attack event.

The state written to the bank at stage 4 for entity  $v$  is the state read at stage 2 the next time  $v$  participates in any event. This creates a recurrent information pathway across entities and across time, with memory bounded at  $\mathcal{O}(|\mathcal{V}| \times d)$  independent of the number of events processed. Training uses Truncated Backpropagation Through Time with detachment at chunk boundaries (Section 4.7).

### 4.2 Event Encoding and Process Identity Modulation

Each audit event  $e_t$  carries three categories of raw input: a discrete event type (one of  $\sim 40$  CDM event types such as EVENT\_READ, EVENT\_MMAP, EVENT\_CONNECT), a vector of continuous features  $\mathbf{x}_{\text{cont}} \in \mathbb{R}^{n_c}$  (byte count, flags, temporal inter-arrival statistics), and the filesystem path of the subject process (e.g., /usr/bin/bash, /home/admin/clean). The event encoder projects these into a dense representation  $\mathbf{f}_e \in \mathbb{R}^{2d}$  that serves as input to all downstream stages.

**Event type and continuous features.** The event type is mapped to a learned embedding  $\mathbf{e}_{\text{type}} = \text{Embed}_{\text{type}}(\text{type}(e_t)) \in \mathbb{R}^d$ . The continuous features are projected via a linear layer  $\mathbf{c} = \mathbf{W}_c \mathbf{x}_{\text{cont}} + \mathbf{b}_c \in \mathbb{R}^d$ .

**Gated process identity modulation.** Integer entity identifiers assigned during preprocessing have no semantic content: the same malware binary /home/admin/clean receives different integer IDs in different campaigns because the entity vocabulary is constructed per-dataset. To provide a semantic signal that generalizes across campaigns, we embed the subject’s process path via a learned lookup  $\mathbf{p}_{\text{raw}} =$



$\text{Embed}_{\text{proc}}(\text{path}(v_{\text{subj}})) \in \mathbb{R}^{64}$  from a vocabulary of known process names (defined in Section ??). This is projected to model dimension  $\mathbf{p} = \mathbf{W}_p \mathbf{p}_{\text{raw}} \in \mathbb{R}^d$ .

Process identity is injected as a *gated additive residual* on the event type embedding rather than via concatenation:

$$\mathbf{e}_{\text{mod}} = \mathbf{e}_{\text{type}} + g_p \cdot \mathbf{p} \quad (5)$$

where  $g_p = \sigma(\gamma_p)$  is a scalar gate controlled by a single learnable parameter  $\gamma_p$ , initialized to  $-2.0$  so that  $g_p \approx 0.12$  at the start of training. Process identity is injected into the event type embedding rather than the continuous features because the process path modulates event semantics (e.g., an `EVENT_MMAP` by `firefox` carries different contextual meaning than one by `pulseaudio`), whereas continuous features measure process-invariant quantities like byte counts. The near-zero gate initialization forces the model to first learn from event-type and continuous features before the process identity signal is mixed in. Without gating, the additive process signal is unconstrained at initialization, creating a gradient shortcut through process identity that bypasses event-level discrimination. We confirm this failure mode empirically in Section 5.5.

**Identity dropout.** During training, each event’s process identity is independently replaced with an unknown token (index 0) with probability  $\rho = 0.3$ :

$$\text{path}(v_{\text{subj}}) \leftarrow \begin{cases} \text{path}(v_{\text{subj}}) & \text{with probability } 1 - \rho \\ \text{<unk>} & \text{with probability } \rho \end{cases} \quad (6)$$

This prevents the classifier from relying on process identity as a sufficient detection signal. The model must maintain a working detection pathway through event-type and entity-state features alone, using process identity as a supplementary modulation rather than a shortcut.

**Final event representation.** The three components are combined and normalized:

$$\mathbf{f}_e = \text{LayerNorm}([\mathbf{e}_{\text{mod}} \parallel \mathbf{c}]) \in \mathbb{R}^{2d} \quad (7)$$

where  $\parallel$  denotes concatenation. The event type (modulated by process identity) occupies the first  $d$  dimensions; continuous features occupy the second  $d$  dimensions.

### 4.3 Entity State Bank

The `EntityStateBank` stores two tensors as registered buffers (not learnable parameters): a state matrix  $\mathbf{S} \in \mathbb{R}^{|\mathcal{V}| \times d}$ , where row  $\mathbf{S}[v]$  holds the current state  $s_v$  of entity  $v$ , and a last-seen timestamp vector  $\boldsymbol{\tau} \in \mathbb{R}^{|\mathcal{V}|}$ , where  $\tau[v]$  records the most recent time entity  $v$  participated in any event. Both tensors are initialized to zero (states) and  $-1$  (timestamps, indicating unseen) at the start of each training epoch and persist across event chunks within the epoch. Resetting at epoch boundaries prevents the model from overfitting to the specific

state trajectory accumulated in earlier epochs, while intra-epoch persistence provides the long-range context required by the thesis.

**Memory bound.** The bank requires  $|\mathcal{V}| \times (d + 1)$  floating-point values. For the Theia dataset ( $|\mathcal{V}| \approx 7.1 \times 10^6, d = 128$ ), this amounts to 3.6 GB in `float32`, fitting within a single GPU.

**State retrieval and elapsed time.** For an incoming event  $e_t$  with entity slots  $(v_{\text{subj}}, v_{\text{obj}}, v_{\text{obj2}})$ , the model gathers a state tensor  $\mathbf{S}_e \in \mathbb{R}^{3 \times d}$  by indexing into  $\mathbf{S}$ . A validity mask  $\mathbf{m} \in \{0, 1\}^3$  is set to 0 for absent entities: size-2 hyperedges store  $v_{\text{obj2}} = -1$ , which is clamped to index 0 during retrieval. The mask ensures that the spuriously read state at index 0 contributes zero to both the aggregation computation and the write-back path, preventing corruption of entity 0’s state.

The elapsed time for each entity is computed as:

$$\Delta t_v = \max(0, t_{\text{curr}} - \tau[v]) \quad (8)$$

where  $\Delta t_v = 0$  for previously unseen entities ( $\tau[v] = -1$ ). The floor at zero guards against non-monotonic timestamps from intra-shard reordering. All timestamps are converted to seconds relative to a global reference  $t_0$  (the nanosecond timestamp of the first event in the training set):

$$t_{\text{curr}} = (\text{timestamp\_nanos}(e_t) - t_0) / 10^9 \quad (9)$$

All splits (train, validation, test) share the same  $t_0$  so that  $\Delta t_v$  remains valid across the train-to-test boundary during warm-start evaluation (Section 4.7).

The elapsed time is passed to downstream stages in log-scaled form as  $\log(1 + \Delta t_v)$  to compress the dynamic range: inter-event gaps in the Theia dataset span from microseconds to hours.

**Hub-entity normalization.** High-degree entities (e.g., `init` with PID 1, which participates in over  $1.5 \times 10^5$  events in the Theia dataset) accumulate state vectors with unbounded norms through repeated SSM updates. Without normalization, the key projections in the aggregation stage (Section 4.4) produce attention scores that overflow `float32` softmax ( $\exp(x)$  for  $x > 88$ ). To prevent this, retrieved states are normalized via layer normalization before entering the aggregation stage. This normalization is applied only to the retrieved copy  $\hat{\mathbf{S}}_e$ ; the bank  $\mathbf{S}$  retains unnormalized states:

$$\hat{\mathbf{S}}_e = \text{LayerNorm}(\mathbf{S}_e) \odot \mathbf{m} \quad (10)$$

The validity mask  $\mathbf{m}$  is applied after normalization to ensure absent entity slots contribute zero to downstream computations. All subsequent references to entity states within a single forward pass refer to  $\hat{\mathbf{S}}_e$ .

**State write-back.** After the SSM state update (Section 4.5), the updated states are scattered back into  $\mathbf{S}$  via in-place indexed assignment, and  $\boldsymbol{\tau}$  is updated with  $t_{\text{curr}}$ . Only valid entities (those with  $\mathbf{m}_v = 1$ ) are written. To prevent numerical corruption from propagating across chunks, NaN values in the updated states are replaced with zero before write-back.

#### 4.4 Hyperedge Aggregation ( $\mathcal{V} \rightarrow \mathcal{E}$ )

Given the normalized entity states  $\hat{\mathbf{S}}_e$  (Equation 10) and the event representation  $\mathbf{f}_e$  (Equation 7), the aggregation stage produces a single hyperedge representation  $\mathbf{x}_e \in \mathbb{R}^d$  that fuses the states of the 2–3 participating entities, conditioned on the event semantics. The aggregation determines *how much* each entity’s accumulated history contributes to the current event’s representation, which directly controls the information available to the classifier.

**Role embeddings.** Each entity slot is assigned a learned role embedding  $\mathbf{r}_v \in \mathbb{R}^d$  from a 3-entry lookup table (Subject, Primary Object, Secondary Object). Role embeddings are necessary because the aggregation operates on an unordered set of states; without positional role information, the attention mechanism cannot distinguish the subject from the object, discarding the directed structure of the event.

**Entity descriptors.** For each entity  $v \in V_{e_t}$ , we construct a descriptor  $\mathbf{h}_v$  by concatenating the normalized state, role embedding, and log-scaled elapsed time:

$$\mathbf{h}_v = [\hat{\mathbf{s}}_v \parallel \mathbf{r}_v \parallel \log(1 + \Delta t_v)] \in \mathbb{R}^{2d+1} \quad (11)$$

**Dynamic attention.** The event features  $\mathbf{f}_e$  serve as the query, and the entity descriptors serve as keys and values. This conditions the aggregation on the event type: for an `EVENT_WRITE`, the object (file being written to) may carry more discriminative state than the subject, while for an `EVENT_MMAP`, the secondary object (memory region) carries the signal identified in Section 2.2. The attention computation is:

$$\mathbf{q} = \mathbf{W}_q \mathbf{f}_e \in \mathbb{R}^d \quad (12)$$

$$\mathbf{k}_v = \mathbf{W}_k \mathbf{h}_v \in \mathbb{R}^d \quad (13)$$

$$\mathbf{v}_v = \mathbf{W}_v \mathbf{h}_v \in \mathbb{R}^d \quad (14)$$

where  $\mathbf{W}_q \in \mathbb{R}^{d \times 2d}$ ,  $\mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{d \times (2d+1)}$ . Attention weights are computed via masked scaled dot-product. For each entity slot  $v$ , the raw score is:

$$s_v = \mathbf{q}^\top \mathbf{k}_v / \sqrt{d} \quad (15)$$

Invalid slots ( $\mathbf{m}_v = 0$ ) have their scores set to  $-\infty$  before softmax, so that  $\exp(-\infty) = 0$  in both numerator and denominator:

$$\alpha_v = \begin{cases} \frac{\exp(s_v)}{\sum_{u: \mathbf{m}_u=1} \exp(s_u)} & \text{if } \mathbf{m}_v = 1 \\ 0 & \text{if } \mathbf{m}_v = 0 \end{cases} \quad (16)$$

As a numerical safeguard, any NaN values in the attention weights (which can arise from degenerate events where all slots are masked, producing 0/0 in softmax) are replaced with zero. The hyperedge representation is then:

$$\mathbf{x}_e = \mathbf{W}_{\text{out}} \sum_{v \in V_{e_t}} \alpha_v \mathbf{v}_v \in \mathbb{R}^d \quad (17)$$

where  $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times d}$  is a linear output projection. The role embeddings  $\mathbf{r}_v$  are retained and passed to the SSM update stage (Section 4.5), where they ensure role-specific state propagation.

#### 4.5 Selective SSM State Update ( $\mathcal{E} \rightarrow \mathcal{V}$ )

Given the hyperedge representation  $\mathbf{x}_e$  (Equation 17) and the role embeddings  $\mathbf{r}_v$  from the aggregation stage, the SSM updater computes a new state  $\mathbf{s}_v^+$  for each participating entity  $v \in V_{e_t}$ . This is the mechanism through which accumulated behavioral context propagates across entities and across time: the updated state encodes both the entity’s prior history (via the decay term) and the current event’s contribution (via the input term).

**State decay matrix.** The diagonal state decay matrix  $A \in \mathbb{R}^d$  is stored in log-space as a learnable parameter  $\mathbf{A}_{\log} \in \mathbb{R}^d$  and recovered as:

$$A = -\exp(\text{clamp}(\mathbf{A}_{\log}, -4, 4)) \quad (18)$$

All entries of  $A$  are strictly negative, ensuring that states decay over time in the absence of new input.  $\mathbf{A}_{\log}$  is initialized with  $\log(\text{linspace}(0.1, 2.0, d))$ , producing initial decay rates that span  $A_j \in [-2.0, -0.1]$ . This multi-timescale initialization assigns different state dimensions different decay rates: slow-decaying dimensions ( $A_j \approx -0.1$ ) retain information over many events for long-range dependencies, while fast-decaying dimensions ( $A_j \approx -2.0$ ) respond to recent activity. Clamping  $\mathbf{A}_{\log}$  to  $[-4, 4]$  prevents numerical overflow during exponentiation.

**Role-specific input projection.** The input matrix  $B$  is computed from the concatenation of the hyperedge representation  $\mathbf{x}_e$  and the entity’s role embedding  $\mathbf{r}_v$ :

$$B_v = \mathbf{W}_B [\mathbf{x}_e \parallel \mathbf{r}_v] \in \mathbb{R}^d \quad (19)$$

where  $\mathbf{W}_B \in \mathbb{R}^{d \times 2d}$ . Including the role embedding produces different  $B_v$  values for the subject, primary object, and secondary object of the same event. Without this, all three entities receive identical update directions from  $\mathbf{x}_e$ , causing their states to converge and destroying the directed graph structure that distinguishes subjects from objects.

**Time-aware discretization.** The continuous-time SSM parameters ( $A, B_v$ ) are discretized using an input-dependent step size  $\Delta_v \in \mathbb{R}^d$ :

$$\Delta_v = \text{clamp}(\text{softplus}(\mathbf{W}_\Delta [\mathbf{x}_e \parallel \mathbf{r}_v]) \cdot (1 + \lambda \log(1 + \Delta t_v)), \max = 5) \quad (20)$$

where  $\mathbf{W}_\Delta \in \mathbb{R}^{d \times 2d}$  with bias initialized to  $-2.0$  (producing initial step sizes of  $\text{softplus}(-2) \approx 0.13$ ),  $\lambda = 0.1$ , and  $\Delta t_v$  is the elapsed time from Section 4.3. The multiplicative time scaling  $(1 + 0.1 \cdot \log(1 + \Delta t_v))$  applies a gentle correction (approximately 1.0 to 1.7 $\times$  over the observed range) that widens the discretization step for entities that have been inactive, allowing the decay to reflect elapsed wall-clock time without

the gradient instability that a direct multiplicative  $\Delta t_v$  scaling would cause. Clamping at 5 prevents numerical overflow in the subsequent exponentiation.

The discretized parameters are:

$$\bar{A}_v = \exp(\Delta_v \odot A) \in (0, 1)^d \quad (21)$$

$$\bar{B}_v = \Delta_v \odot B_v \quad (22)$$

where  $\bar{A}_v$  controls per-dimension retention of the prior state (values near 1 retain, values near 0 forget), and  $\bar{B}_v$  scales the input contribution. The raw SSM update is:

$$\tilde{s}_v = \bar{A}_v \odot \hat{s}_v + \bar{B}_v \odot \mathbf{x}_e \quad (23)$$

**Gated residual connection.** A learned gate  $\mathbf{g}_v \in (0, 1)^d$  blends the SSM output with the prior state:

$$\mathbf{g}_v = \sigma(\mathbf{W}_g[\mathbf{x}_e \parallel \mathbf{r}_v]) \quad (24)$$

$$\mathbf{s}_v^+ = \mathbf{g}_v \odot \tilde{s}_v + (1 - \mathbf{g}_v) \odot \hat{s}_v \quad (25)$$

where  $\mathbf{W}_g \in \mathbb{R}^{d \times 2d}$  with bias initialized to  $-3.0$ , so that  $\mathbf{g}_v \approx 0.04$  at the start of training. This near-closed initialization is structurally necessary: at random initialization, the SSM parameters ( $A, B$ ) produce arbitrary state updates, and an open gate would immediately overwrite entity states with noise. Initializing the gate near zero forces the model to actively learn *when* to propagate state, preventing over-smoothing at initialization. The gate opens during training as the SSM parameters become meaningful.

## 4.6 Classification Head

The final stage determines whether the event  $e_t$  is part of an attack sequence by evaluating both its isolated semantics and the accumulated behavioral state of its participating entities.

**Gradient routing via post-update states.** The classifier operates on the *post-update* states  $\mathbf{s}_v^+$  computed in Section 4.5, not the pre-update states retrieved from the bank. This routing is structurally critical for end-to-end training. The bank’s states are treated as detached constants during the forward pass; if the classifier read directly from the bank, the computational graph would end, cutting off all gradient flow to the SSM updater (which produced the new state) and the hyper-edge aggregator (which produced the SSM input). By classifying the post-update state, the gradient propagates backward through the entire forward pass.

**Validity-masked mean pooling.** The post-update states are pooled into a single event-level context vector  $\bar{\mathbf{s}} \in \mathbb{R}^d$ . This requires masking out absent secondary objects (where  $\mathbf{m}_v = 0$ ):

$$\bar{\mathbf{s}} = \text{LayerNorm} \left( \frac{1}{\sum_v \mathbf{m}_v} \sum_{v \in V_{e_t}} \mathbf{m}_v \cdot \mathbf{s}_v^+ \right) \quad (26)$$

Masking before summation is required because the SSM update (Equation 23) produces a non-zero update even for the null entity slot (driven by the event representation  $\mathbf{x}_e$  through the role projection  $\bar{B}_v$ ). Without the mask, the null slot would corrupt the mean. The pooled state is layer-normalized to stabilize the classifier input.

**Detection probability.** The pooled state is concatenated with the projected event features  $\mathbf{W}_p \mathbf{f}_e$  (where  $\mathbf{W}_p \in \mathbb{R}^{d \times 2d}$ ). The probability  $\hat{y}_t \in [0, 1]$  that  $e_t$  is an attack event is computed via a Multi-Layer Perceptron (MLP):

$$\hat{y}_t = \sigma(\text{MLP}([\bar{\mathbf{s}} \parallel \mathbf{W}_p \mathbf{f}_e])) \quad (27)$$

The MLP consists of three linear layers with intermediate GELU activations and Dropout ( $\rho = 0.1$ ) after the first layer, projecting dimensions from  $2d \rightarrow d \rightarrow d/2 \rightarrow 1$ . A Binary Cross-Entropy (BCE) loss is computed against the ground-truth event label  $y_t \in \{0, 1\}$ .

## 4.7 Training: TBPTT and Continuous Warm-Start

HyperMamba requires training techniques distinct from stateless NIDS models because entity states persist across the event sequence.

**Truncated Backpropagation Through Time (TBPTT).** To process continuous event streams without exhausting memory, events are partitioned chronologically into chunks of size  $C' = 4096$ . For each chunk, the model executes the forward pass (Sections 4.2–4.6) and computes the BCE loss. Crucially, the updated states written to the EntityStateBank are retained for the next chunk, but their computational graphs are detached via ‘.detach()’. This TBPTT strategy allows *forward* context to persist indefinitely across the entire dataset, accumulating long-range taint signals, while bounding *backward* gradient computation to  $\mathcal{O}(C)$ .

**Optimization and stability.** The model is optimized using Adam with a Cosine Annealing learning rate schedule. To handle the severe class imbalance inherent to audit streams, the BCE loss is weighted by the ratio of negative to positive examples in the training set (capped at 30.0 to prevent gradient explosion). Because SSMs operating on discrete streams can occasionally produce numerical instability, chunks resulting in NaN losses or NaN gradients are skipped, and the state bank is immediately detached to prevent corruption of subsequent chunks. Gradient norms are additionally clipped to 1.0.

**Epoch boundary reset.** The EntityStateBank is completely zeroed out at the start of every training epoch. Without this reset, the model would begin epoch  $k$  (training on the first chronological shard) using the future entity states accumulated at the end of epoch  $k - 1$  (after evaluating on the validation shards). This would create a severe temporal data leak. Resetting ensures each epoch learns to accumulate state strictly from the past.

**Continuous warm-start evaluation.** In a real-world deployment, an NIDS runs continuously; entity states are never “cold.” To accurately evaluate detection capability on a test shard, we must simulate this continuous deployment. Before test evaluation, the model executes a forward-pass-only *warmup* (with gradients and metric collection disabled) through all prior training and validation shards in strict chronological order to populate the `EntityStateBank`. The test set is then evaluated using these warmed states. Evaluating a stateful model from a cold start systematically underestimates its capability to detect multi-stage attacks that began prior to the test window.

**Threshold selection.** In accordance with rigorous evaluation standards for security machine learning [?], the decision threshold is selected dynamically to maximize the F1-score *exclusively* on the validation split. This fixed threshold is then applied to the test split, completely preventing test-set threshold leakage.

## 5 Evaluation

### 5.1 Experimental Setup

**Dataset and label definition.** We evaluate HyperMamba on the DARPA Transparent Computing (TC) Engagement 3 Theia dataset, a standard benchmark for provenance-based NIDS. After ingestion, the dataset comprises 25 chronological shards containing roughly 105 million events. We target the *crossprocess* label: events executed by the multi-stage child processes spawned by an initial compromised entry-point process. The entry-point process itself is excluded from this label, forcing the detection system to identify the malicious activity of the later stages (e.g., a dropped script connecting to a C2 server) rather than simply flagging the initial anomalous behavior of the compromised application.

**Splits.** The evaluation is strictly chronological to prevent temporal data leakage. The data encompasses two distinct multi-stage APT campaigns. Campaign 1 spans shards 0–10, and Campaign 2 spans shards 12–24. To evaluate both same-campaign continuation and cross-campaign generalization, we define two evaluation regimes:

- **Mixed-campaign (full):** Both campaigns are present in the training set. We assign shards {0..7, 12..19} to training, shards {8, 9, 20, 21} to validation, and shards {10, 22..24} to testing. This matches the standard evaluation protocol of our baselines.
- **Cross-campaign (cross):** Tests generalization to entirely unseen attack strategies. The model is trained exclusively on Campaign 1 (train shards {0..8}, validation shards {9, 10}) and tested exclusively on Campaign 2 (test shards {12..24}).

**Baselines.** We compare HyperMamba against state-of-the-art provenance-based NIDS, positioning each by its fundamental detection granularity:

- **KAIROS** [?]: Operates at the graph level, assigning anomaly scores to entire macroscopic provenance graphs.
- **MAGIC** [?]: Operates at the graph level, detecting attacks via reconstruction error of structural subgraphs.
- **FLASH** [?]: Operates at the node level, flagging anomalous processes or files.
- **ThreaTrace** [?]: Operates at the node level, classifying entities within the provenance graph.

Crucially, none of these baselines are natively capable of *event-level* detection, which is the finest granularity and the primary output of HyperMamba.

**Metrics and threshold methodology.** We report Area Under the Precision-Recall Curve (AUPRC) as the primary threshold-independent metric, given the severe class imbalance of audit streams. To compute threshold-dependent metrics (F1-score and False Positive Rate), we adhere to the strict guidelines for security machine learning proposed by Arp et al. [?]: thresholds are selected dynamically from the validation predictions using the F1-maximizing operating point, and these fixed values are subsequently applied to the held-out test data. This strictly prevents test-set threshold optimization leakage. Metrics are reported at both the event level (for HyperMamba) and the node level (for HyperMamba and baselines).

### 5.2 Main Results (C4)

We report the detection performance of HyperMamba against the state-of-the-art provenance baselines in Table 1.

**Granularity incomparability.** We emphasize that direct numerical comparison between HyperMamba’s event-level metrics and the baselines’ node- or graph-level metrics is structurally invalid. Detecting one anomalous node among  $10^4$  processes is a fundamentally different, and easier, statistical task than classifying the exact 10 attack events out of the  $10^5$  events that node executes. We report both event-level and aggregated node-level metrics to demonstrate that HyperMamba remains competitive at the coarse node level while uniquely providing the finer event-level forensic precision.

**Same-campaign vs. Cross-campaign detection.** We decompose HyperMamba’s performance into two evaluation regimes. Under the *same-campaign* continuation setting (evaluating on unseen later stages of campaigns present in the training set), HyperMamba achieves a remarkable event-level AUPRC of 0.86 with a near-zero False Positive Rate of 0.000010. At an operational scale of 100 million events, this translates to roughly 1,000 false alerts—a manageable load



Table 1: Detection Performance on DARPA TC E3 Theia. HyperMamba is the only system capable of event-level detection. Baseline numbers are reported at their native, coarser granularities.

| Model                       | Granularity  | Event AUPRC | Event F1 | Event FPR | Node AUPRC | Node F1 |
|-----------------------------|--------------|-------------|----------|-----------|------------|---------|
| KAIROS                      | Graph        | -           | -        | -         | -          | -       |
| MAGIC                       | Graph        | -           | -        | -         | -          | -       |
| FLASH                       | Node         | -           | -        | -         | -          | -       |
| ThreaTrace                  | Node         | -           | -        | -         | -          | -       |
| HyperMamba (Same-Campaign)  | Event & Node | 0.86        | -        | 0.000010  | -          | -       |
| HyperMamba (Cross-Campaign) | Event & Node | 0.72        | -        | 0.000195  | -          | -       |

Table 2: Ablation: Impact of Persistent Entity State Propagation

| Model                 | Train AUPRC | Train F1 | Test AUPRC | Test F1 |
|-----------------------|-------------|----------|------------|---------|
| HyperMamba (Full)     | 0.86        | -        | 0.73       | -       |
| HyperMamba (no_state) | 0.51        | -        | 0.33       | -       |

for security analysts, contrasting sharply with the alert fatigue caused by systems with higher FPRs.

Under the stringent *cross-campaign* generalization setting (training strictly on Campaign 1 and evaluating strictly on Campaign 2), the event-level AUPRC remains strong at 0.72, with an FPR of 0.000195. This demonstrates that the SSM state propagation mechanism learns generalizable attack behaviors rather than simply memorizing the specific topology of the training campaign.

### 5.3 Ablation: Entity State Propagation (C2)

The central thesis of HyperMamba is that persistent accumulated entity state—not isolated event features—is the primary mechanism allowing detection of stealthy crossprocess attacks. To empirically prove this (Contribution 2), we compare the full HyperMamba architecture against a stateless ablation (no\_state).

**Ablation design.** The no\_state variant retains the complete architecture: the event encoder, the hyperedge aggregator, and the classifier head remain identical. The only change is structurally preventing long-range state accumulation by resetting the EntityStateBank at every chunk boundary, rather than detaching and persisting the states (Section 4.7). The model is forced to evaluate each chunk using only the local event features and intra-chunk context.

**Results.** As shown in Table 2, removing persistent state propagation causes a catastrophic collapse in detection capability. Test AUPRC drops from 0.73 to 0.33. Crucially, the training AUPRC also collapses from 0.86 to 0.51. The failure to fit even the training distribution confirms the structural argument from Section 3: crossprocess attack events are not identifiable from local event features alone. Without the accumulated history of the entry-point processes that spawned them, the child processes executing the attack are statistically

Figure 1: Training and validation AUPRC curves per epoch. The no\_state ablation diverges immediately, failing to find a discriminative signal.

indistinguishable from benign system activity. This empirically validates the core thesis of HyperMamba: operating at the event-level requires persistent entity state, and prior stateless or window-based approaches are structurally incapable of matching this precision.

The training curves (Figure 1) further illustrate this mechanism. The full model’s validation AUPRC steadily improves as the SSM parameters learn to route and retain long-range signals. In contrast, the no\_state variant diverges almost immediately, completely failing to find a discriminative boundary using isolated event features.

### 5.4 Ablation: Cross-Entity Aggregation

While Section 5.3 establishes that state persistence is mandatory, we must also evaluate whether *sharing* that state across entities within an event is necessary. To determine if the hyperedge structure (Section ??) contributes beyond simple isolated per-entity state tracking (Contribution 1), we evaluate the no\_cross\_entity ablation.

**Ablation design.** In this variant, the hyperedge aggregator is disabled. When processing an event, the SSM updates the state of the subject entity using only the subject’s prior state and the event features. The states of the secondary object entities (e.g., the target file or socket) do not influence the subject’s update, nor do they receive the subject’s accumulated taint.

Table 3: Ablation: Impact of Cross-Entity Hyperedge Aggregation

| Model                        | Event AUPRC | Event F1 |
|------------------------------|-------------|----------|
| HyperMamba (Full)            | 0.86        | -        |
| HyperMamba (no_cross_entity) | -           | -        |

**Results.** As shown in Table 3, disabling cross-entity aggregation degrades detection performance. While isolated per-entity state tracking (no\_cross\_entity) outperforms the

purely stateless baseline, it fails to match the full hypergraph model. This confirms that tracking the topological flow of operations—such as a subject process inheriting the anomalous taint of a maliciously modified object file—provides a critical secondary signal for detecting sophisticated multi-stage campaigns.

## 5.5 Ablation: Process Identity Modulation (C3)

While state propagation enables the model to connect related events within a campaign, generalizing that detection capability to entirely unseen campaigns requires stable semantic anchors. As described in Section 4.2, integer entity IDs carry no meaning across campaigns—the identifier  $e_1$  might represent ‘bash’ in Campaign 1 but a temporary file in Campaign 2. To prove that our gated process identity modulation is the critical mechanism for cross-campaign generalization (Contribution 3), we ablate the process path embeddings on the strict *cross-campaign* evaluation split.

Table 4: Ablation: Process Identity Modulation on Cross-Campaign Split

| Model                            | Node AUPRC |
|----------------------------------|------------|
| HyperMamba (Full)                | 0.89       |
| HyperMamba (no_process_identity) | -          |

**Results.** Table 4 demonstrates the impact on cross-campaign generalization. Without the semantic process embeddings modulating the event encoder, the model is forced to rely purely on temporal patterns and event types. Because the underlying integer IDs have zero semantic overlap between the training and test campaigns, the model’s ability to identify structurally similar attack behaviors degrades significantly. The process identity embeddings provide the essential invariant signal—allowing the model to recognize that an anomalous pattern executed by ‘bash’ is fundamentally different from the same pattern executed by ‘nginx’, regardless of the specific campaign context.

## 5.6 Computational Cost

A critical requirement for any NIDS is the ability to operate in real-time without falling behind the host’s audit log generation rate. We benchmark the computational overhead of HyperMamba, summarizing the results in Table 5.

**Throughput.** The DARPA TC E3 dataset was generated at a peak rate of approximately 10,000 events per second per host. As shown in Table 5, HyperMamba achieves a training throughput of 734K events/sec and an inference throughput of 1.94M events/sec on a single GPU. Even when deployed in a resource-constrained, CPU-only environment, HyperMamba

Table 5: Computational Cost and Throughput

| Metric                                                 | Value           |
|--------------------------------------------------------|-----------------|
| Parameters (HyperMamba)                                | 1.17M           |
| Parameters (GRU-TGN Baseline)                          | 1.31M           |
| State Memory ( $7.1\text{M entities} \times d = 256$ ) | 7.2 GB          |
| Training Throughput (GPU)                              | 734K events/sec |
| Inference Throughput (GPU)                             | 1.9M events/sec |
| Inference Throughput (CPU)                             | 70K events/sec  |

processes 70K events/sec— $7\times$  faster than the peak generation rate. On a GPU, the pipeline operates at  $73\times$  the peak ingestion rate during training and an overwhelming  $194\times$  the ingestion rate during inference, guaranteeing that the model will never bottleneck the audit stream.

**Memory and parameters.** Despite operating on a vast continuous hypergraph, HyperMamba is highly parameter-efficient, requiring only 1.17M trainable parameters (compared to 1.31M for a standard GRU-based Temporal Graph Network baseline). The primary memory footprint is the EntityStateBank. For the Theia dataset’s 7.1 million unique entities at  $d = 256$  (using 32-bit floats), the bank requires 7.2 GB of memory. This easily fits in the RAM of standard enterprise workstations or a single commodity GPU, demonstrating that event-level stateful tracking is achievable without distributed infrastructure.

## 6 Discussion

We state the limitations of this work as factual constraints of the current evaluation.

**Dataset scope.** The evaluation is conducted exclusively on the DARPA TC Engagement 3 Theia dataset under the crossprocess label definition. Generalization to other audit frameworks (sysdig, eBPF), operating systems beyond Linux/FreeBSD, or enterprise environments with substantially different workload distributions is not evaluated.

**Label coverage.** The crossprocess label captures multi-stage child-process attack chains but does not cover all attack types. Data-only attacks that never spawn new processes (e.g., exploits that corrupt in-memory data structures without executing new binaries) fall outside this label definition and are not addressed by the current evaluation.

**Entity churn and LRU eviction.** The EntityStateBank architecture supports LRU eviction to bound memory under sustained entity churn. However, this eviction mechanism is not stress-tested in this evaluation. The Theia dataset’s 7.1 million unique entities fit comfortably in the bank at  $d = 256$ ; deployments with significantly larger entity populations may require eviction, and the impact of eviction on detection performance is unknown.

**Adversarial robustness.** The primary evaluation assumes a black-box attacker (Section 3.1). A white-box adversary

Table 6: Positioning of HyperMamba against provenance-based NIDS on three axes.

| System     | Granularity | State      | Supervision     |
|------------|-------------|------------|-----------------|
| KAIROS     | Graph       | Snapshot   | Self-supervised |
| MAGIC      | Graph       | Snapshot   | Self-supervised |
| FLASH      | Node        | Snapshot   | Self-supervised |
| ThreaTrace | Node        | Snapshot   | Supervised      |
| HyperMamba | Event       | Persistent | Supervised      |

who understands the SSM state propagation mechanism could potentially craft evasion strategies — for example, injecting sequences of benign events to dilute a compromised entity’s accumulated state before executing attack operations. The robustness of HyperMamba against such targeted state-dilution attacks is not evaluated.

**Payload-based attacks.** HyperMamba operates exclusively on structured audit metadata (event types, byte counts, timestamps). It does not inspect network packet payloads or file contents. Attacks that are distinguishable only by payload content (e.g., SQL injection within an HTTP request body) are outside the scope of this system.

**Single-host scope.** The current evaluation considers audit streams from a single monitored host. Multi-host lateral movement — where an attacker pivots across machines in a distributed enterprise environment — requires cross-host state correlation, which is not addressed in this work.

## 7 Related Work

We position HyperMamba against prior work on three axes: detection granularity, temporal state mechanism, and supervision regime. Table 6 summarizes the landscape.

**Provenance-based NIDS.** KAIROS [?] constructs temporal provenance graphs and assigns anomaly scores at the graph level via reconstruction-based learning. MAGIC [?] similarly operates at the graph level, detecting attacks through structural subgraph reconstruction error. FLASH [?] provides node-level alerts by identifying anomalous processes or files within the provenance graph. ThreaTrace [?] classifies individual entities (nodes) using graph neural networks. All four systems model provenance as pairwise graphs, discarding the native multi-object structure of CDM events. More fundamentally, none produce event-level classifications — the finest granularity required for forensic attribution of individual attack steps.

**Temporal state mechanism.** The systems above operate on static or windowed graph snapshots, constructing a new graph for each analysis window and discarding inter-window entity state. HyperMamba maintains persistent entity state vectors across the entire event stream via a Selective SSM, enabling the model to accumulate behavioral context over arbitrarily long horizons. The ablation in Section 5.3 demon-

strates that this persistent state — not event features alone — is the primary detection mechanism.

**Supervision trade-off.** MAGIC and KAIROS are self-supervised: they train exclusively on benign data and flag statistical deviations, avoiding the need for attack-labeled training data. HyperMamba is supervised: it requires crossprocess attack labels during training. This is an intentional design choice. Anomaly-based systems operate at graph or window level without event-level attribution because their reconstruction objective does not associate individual events with attack labels. Our supervised formulation requires crossprocess labels but enables event-level detection precision that self-supervised approaches cannot provide. Whether labeled crossprocess data is available in practice depends on the deployment context; we discuss this limitation in Section 6.

**Hypergraph-based intrusion detection.** Several systems have explored hypergraph representations for intrusion detection, though in different domains and with different objectives. LOHA [?] uses reconstruction-based anomaly detection on hypergraphs constructed from network flows. HyperGAT [?] applies hypergraph attention for static community mining. HRNN [?] constructs hyperedges via KNN over network-flow feature vectors. None of these systems use deterministic, event-level hyperedges derived from the audit data model, and none maintain persistent entity state across the stream.

**State Space Models.** The Selective SSM was introduced by Mamba [?] for efficient sequence modeling in NLP and vision, building on the structured SSM foundations of S4 [?] and S6 [?]. HyperMamba adapts the Selective SSM to a fundamentally different setting: rather than processing a single token sequence, the SSM operates on per-entity state vectors within a temporal hypergraph, with the discretization step  $\Delta$  conditioned on entity-specific elapsed time (Section 4.5).

## 8 Conclusion

We presented HyperMamba, a provenance-based NIDS that detects individual crossprocess attack events in continuous audit streams by maintaining persistent entity state via Selective State Space Models. The ablation study demonstrates that this accumulated state is the primary detection mechanism: removing it collapses test AUPRC from 0.73 to 0.33, confirming that crossprocess attack events are not identifiable from isolated event features. On the DARPA TC E3 Theia dataset, HyperMamba achieves event-level AUPRC of 0.86 under same-campaign continuation and 0.72 under cross-campaign generalization — the latter enabled by gated process identity modulation, which provides campaign-invariant semantic anchors — with false positive rates of 0.000010 and 0.000195 respectively, while processing events at  $194\times$  the peak audit ingestion rate. The evaluation is limited to a single dataset family (DARPA TC E3) under the crossprocess label definition; generalization to other audit frameworks and attack taxonomies remains to be validated.